Individual assignment

LMT

Abstract

Este es el abstract.

1 Introduction

Digital electronics are the backbone of the technological society we live in today. This advancement has provided humanity with multiple benefits, ranging from comunications to health care. The evolution of digital electronics has been marked by the devopment of numerous technologies and methodologies that have revolutionized the way we process and transmit information. One of these improvements are the development of combinational logic circuits, which are fundamental components in digital systems. These circuits perform specific logical operations based on the combination of input signals, providng the users with quick and efficient data processing capabilities, whithout the need for memory elements. Through the development of combinational circuits in the laboratory, we are able to understand and develop a deep understanding of the applications of digital electronics in various fields, such as computing, telecommunications, and control systems.

2 Objectives

- Design combinational logic circuits.
- Program and simulate custom combinational logic circuits in VHDL.

To achieve the objectives mentioned above, we must first understand the concepts of combinational logic circuits and the implementation in VHDL programming. First and foremost, combinational logic circuits "are based on present inputs, and produces outputs that directly corresponds to the applied input signals." [1]. This means that the output of the circuit is determined solely by the current state of the inputs, without any memory or feedback elements involved.

3 Methodology

The methodology applied in this laboratory practice was structured into three main phases: logical analysis, simulation in EDA Playground, and physical implementation on the Basys 3 board.

3.1 Logical Analysis Phase

During this first stage, the following activities were carried out:

1. The truth tables and/or the functional requirements for each problem were analyzed and defined.
2. Based on the truth tables or logical functions, the logic for the solution was laid out and optimized.
3. Finally, the obtained results were documented for use in the subsequent simulation and programming phases.

3.2 EDA Playground Simulation Phase

In the EDA Playground digital simulation environment, the following procedures were performed:

1. The rock, paper and scissors game, water pump program, and coin detector circuits were programmed using the VHDL language.
2. Testbenches were developed to verify the behavior of each circuit to prove certain behaviours of the code.
3. The results obtained from the simulations were analyzed and compared with the previously developed truth tables and logic functions, ensuring the correct logical implementation of each design.

3.3 Basys 3 Board Implementation Phase

Finally, the physical implementation of the circuits was carried out on the Basys 3 board, following the steps described below:

1. The code for each circuit was synthesized, and the corresponding bitstream files were generated.

2. Pin constraints (`.xdc` file) were assigned to map the logical signals to the physical components of the board (switches, LEDs, and displays).

3. The designs were loaded onto the Basys 3 board, verifying their correct operation through practical input and output tests.

4 Results

4.1 Rock Paper Scissors Game

For the rock, paper, scissors game, the program was thougt out to have two players and a begin button, where each player could flip switches to select their choce. And once the play switch was raised, leds would light up to indicate the winner of the game. For this circuit, the follwing truth table was developed:

Input				
PB	**JAA**	**JAB**	**JBA**	**JBB**
1	0	0	0	0
1	0	0	0	1
1	0	0	1	0
1	0	0	1	1
1	0	1	0	0
1	0	1	0	1
1	0	1	1	0
1	0	1	1	1
1	1	0	0	0
1	1	0	0	1
1	1	0	1	0
1	1	0	1	1
1	1	1	0	0
1	1	1	0	1
1	1	1	1	0
1	1	1	1	1

Tabla 1: Input Combinations for ent_juego

Output (SAL)		
E	**GJA**	**GJB**
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
1	0	0
0	0	1
0	1	0
0	0	0
0	1	0
1	0	0
0	0	1
0	0	0
1	0	0
0	1	0
1	0	0

Tabla 2: Corresponding Outputs for ent_juego

And based on these truth tables, a code was generated in VHDL to simulate and implement the circuit. The code is shown below:

Listing 1: VHDL Code for Rock Paper Scissors Game

```
library IEEE;
use IEEE.std_logic_1164.all;

-- Entity declaration
entity juego is
port(
        PB : in std_logic;
    JAA, JAB, JBA, JBB : in std_logic;
    SAL : out std_logic_vector(2 downto 0)
);
end juego;

-- Architecture definition
architecture arq_juego of juego is

    signal JGO : std_logic_vector(4 downto 0);

 begin

    JGO <= PB & JAA & JAB & JBA & JBB;

    with (JGO) select
        SAL <=  "100" when "10101",
                "001" when "10110",
                "010" when "10111",
                "010" when "11001",
                "100" when "11010",
                "001" when "11011",
```

```
                "001" when "11101",
                "010" when "11110",
                "100" when "11111",
                "000" when others;


end arq_juego;
```

From which, the following testbench was created to verify the functionality of the code:

Listing 2: VHDL Testbench for Rock Paper Scissors Game

```
library IEEE;
use IEEE.std_logic_1164.all;

entity testbench is
-- empty
end testbench;

architecture tb of testbench is

-- DUT component
component juego is
port(
  PB: in std_logic;
  JAA: in std_logic;
  JAB: in std_logic;
  JBA: in std_logic;
  JBB: in std_logic;
  SAL: out std_logic_vector(2 downto 0));
end component;

signal PB_in, JAA_in, JAB_in, JBA_in,JBB_in: std_logic;
signal SAL_out: std_logic_vector(2 downto 0);

begin

  -- Connect UUT
  UUT: juego port map(PB_in, JAA_in, JAB_in, JBA_in, JBB_in, SAL_out);

  process
  begin
    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '0';
    JBA_in <= '0';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '0';
    JBA_in <= '0';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '0';
    JBA_in <= '1';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '0';
    JBA_in <= '1';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '1';
    JBA_in <= '0';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '1';
    JBA_in <= '0';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '1';
    JBA_in <= '1';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '0';
    JAB_in <= '1';
    JBA_in <= '1';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '1';
    JAB_in <= '0';
    JBA_in <= '0';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '0';
    JAA_in <= '1';
    JAB_in <= '0';
```

```
JBA_in <= '0';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '0';
JBA_in <= '1';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '0';
JBA_in <= '1';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '1';
JBA_in <= '0';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '1';
JBA_in <= '0';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '1';
JBA_in <= '1';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '0';
JAA_in <= '1';
JAB_in <= '1';
JBA_in <= '1';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '0';
JBA_in <= '0';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '1';
```

```
JAA_in <= '0';
JAB_in <= '0';
JBA_in <= '0';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '0';
JBA_in <= '1';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '0';
JBA_in <= '1';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '1';
JBA_in <= '0';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '1';
JBA_in <= '0';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '1';
JBA_in <= '1';
JBB_in <= '0';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '0';
JAB_in <= '1';
JBA_in <= '1';
JBB_in <= '1';
wait for 1 ns;

PB_in <= '1';
JAA_in <= '1';
JAB_in <= '0';
JBA_in <= '0';
JBB_in <= '0';
wait for 1 ns;
```

```
    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '0';
    JBA_in <= '0';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '0';
    JBA_in <= '1';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '0';
    JBA_in <= '1';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '1';
    JBA_in <= '0';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '1';
    JBA_in <= '0';
    JBB_in <= '1';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '1';
    JBA_in <= '1';
    JBB_in <= '0';
    wait for 1 ns;

    PB_in <= '1';
    JAA_in <= '1';
    JAB_in <= '1';
    JBA_in <= '1';
    JBB_in <= '1';
    wait for 1 ns;

    wait;


  end process;

end tb;
```

And the simulation on EDA Waves was generated as follows:

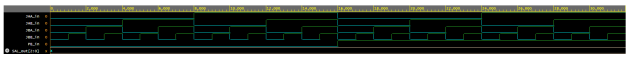

Figure 1: EDA Waves Simulation for Rock Paper Scissors Game

And finally, the circuit was implemented on the Basys 3 board, obtaining the following results:

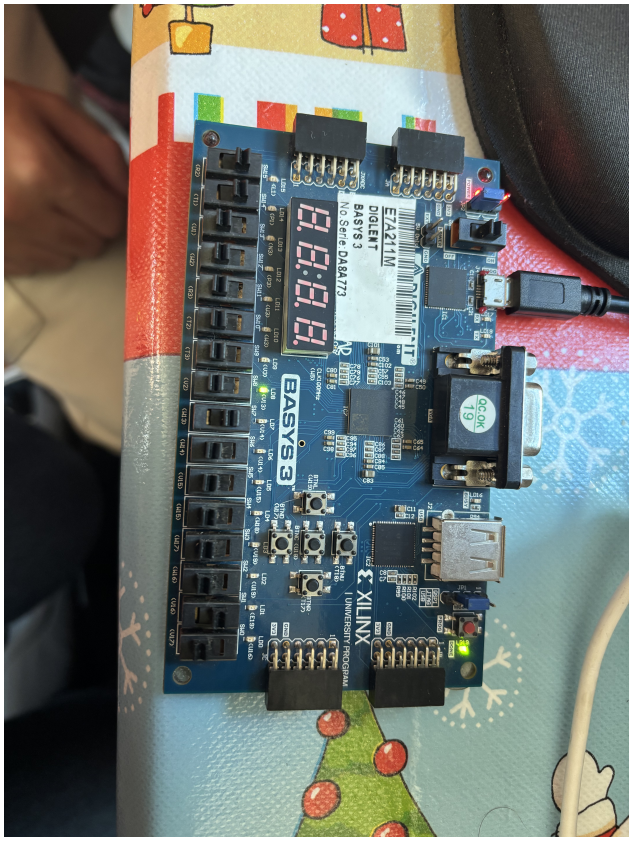

Figure 2: Basys 3 Board Implementation for Rock Paper Scissors Game, A Wins

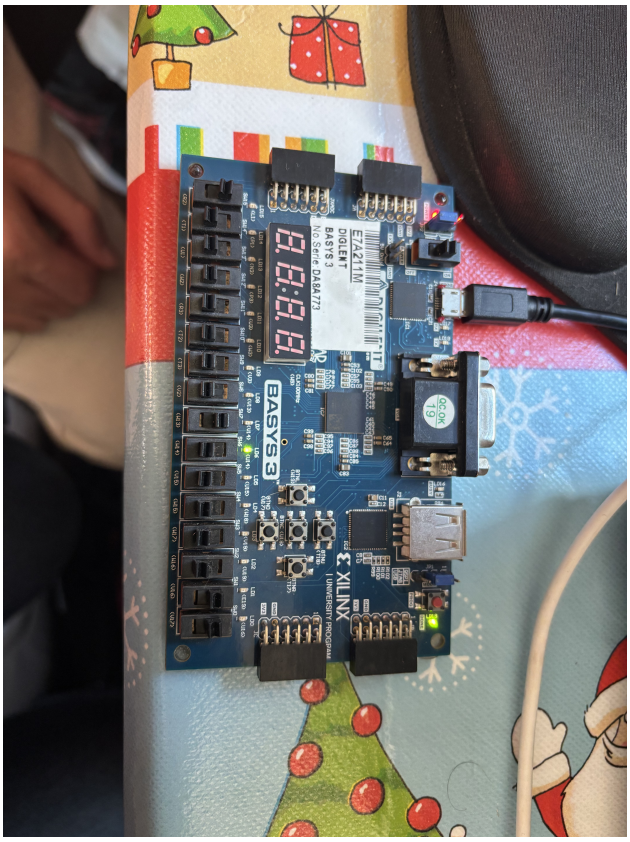

Figure 3: Basys 3 Board Implementation for Rock Paper Scissors Game, B Wins

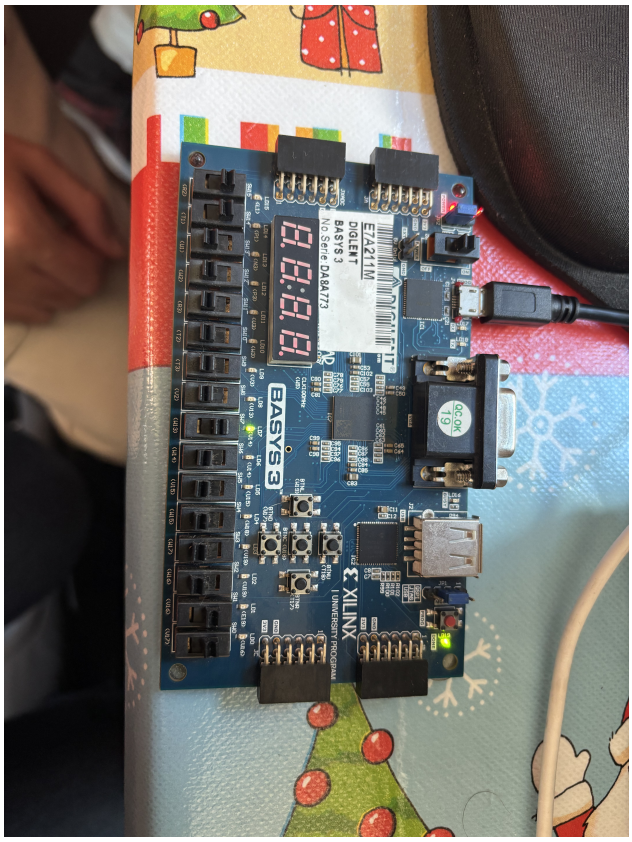

Figure 4: Basys 3 Board Implementation for Rock Paper Scissors Game, Tie

References

[1] L. M. I. L. Joseph. (2025, sep) Combinational logic circuits: The silent power behind modern hardware. [En línea]. Disponible: https://www.linkedin.com/pulse/combinational-logic-circuits-silent-power-behind-modern-joseph-qbzpc